

Processamento de Imagens Médicas

Parte 1: Preparando e Organizando as Imagens

- **Dataset:** BloodMNIST (MedMNIST)
- **Ambiente:** Google Colab (Python 3.10)
- **Bibliotecas:** medmnist, PyTorch, torchvision, NumPy, Matplotlib, scikit-learn
- **Ano:** 2026

1. O que é este projeto?

Este documento mostra como preparamos as imagens do dataset **BloodMNIST** (que faz parte do MedMNIST, um padrão bem conhecido na área médica). Esse dataset traz mais de 17 mil fotos de células do sangue tiradas em microscópio, divididas em 8 tipos diferentes.

O nosso foco aqui foi limpar, padronizar e organizar essas imagens para que os modelos de inteligência artificial consigam aprender com elas da melhor forma possível.

2. Conhecendo os Dados

Visão Geral

As imagens originais são coloridas (RGB) e foram padronizadas pelo MedMNIST no tamanho de 28x28 pixels. O volume total de dados está dividido assim:

- **Total de imagens:** 17.092
- **Treino:** 11.959 imagens (~70%) — usadas para o modelo aprender.
- **Validação:** 1.712 imagens (~10%) — usadas para ajustar o modelo.
- **Teste:** 3.421 imagens (~20%) — usadas para a avaliação final.
- **Licença:** CC BY 4.0

Quais células estão no dataset?

1. **Basófilo (Basophil):** Célula de defesa com pontinhos roxos escuros.
2. **Eosinófilo (Eosinophil):** Combate alergias e parasitas.
3. **Eritroblasto (Erythroblast):** A versão "jovem" do glóbulo vermelho.
4. **Granulócito Imaturo (IG):** Célula de sangue ainda em desenvolvimento.
5. **Linfócito (Lymphocyte):** Peça-chave da nossa imunidade.
6. **Monócito (Monocyte):** Célula que "fagocita" (limpa) invasores no sangue.
7. **Neutrófilo (Neutrophil):** A mais comum, focada em combater bactérias.
8. **Plaqueta (Platelet):** Pedacos de células que ajudam na coagulação.

3. O Passo a Passo do Pré-processamento

Para deixar as imagens no ponto certo, criamos uma sequência de 7 etapas:

1. **Download:** Baixamos o BloodMNIST usando a separação oficial de dados.
2. **Aumento de tamanho:** Mudamos de 28x28 para **64x64 pixels**. Isso ajuda modelos modernos (como ResNet) a enxergarem melhor os detalhes das células. Usamos o comando: `transforms.Resize(64, 64)`.
3. **Conversão para Tensor:** Transformamos as imagens em formatos que o PyTorch entende, mudando os valores dos pixels de 0-255 para uma escala entre 0.0 e 1.0.
4. **Cálculo de Estatísticas:** Descobrimos a média e o desvio padrão das cores das imagens de treino para ajustar o equilíbrio do canal de cores (Vermelho, Verde e Azul).
5. **Normalização:** Aplicamos essas médias para deixar os dados equilibrados, o que faz a IA aprender muito mais rápido.
6. **Variedade de Dados (Augmentação):** Como as células podem aparecer viradas em qualquer direção no microscópio, aplicamos giros aleatórios de até 15° e espelhamento horizontal (com 50% de chance) nas fotos de treino.
7. **Criação dos DataLoaders:** Organizamos tudo em pacotes de 32 imagens (`batch_size=32`) e embaralhamos os dados de treino para o modelo não viciar na ordem das fotos.

4. Como os dados foram divididos

Seguimos a divisão padrão do próprio dataset, garantindo que todas as classes de células apareçam de forma justa em cada grupo:

Grupo	Quantidade	Proporção	Ganha rotação/espelhamento?	É embaralhado?
Treino	11.959	~70%	Sim	Sim
Validação	1.712	~10%	Não	Não
Teste	3.421	~20%	Não	Não
Total	17.092	100%		

5. Boas Práticas que Segguimos

- **Resultados Reprodutíveis:** Fixamos uma "semente" aleatória (`seed=42`). Isso significa que qualquer pessoa que rodar o código vai ter exatamente o mesmo resultado que nós.
- **Sem trapaça (Data Leakage):** As estatísticas de cor foram calculadas *apenas* no grupo de treino. Usar dados de teste para isso seria como dar as respostas da prova antes do exame.
- **Modificações só no treino:** As alterações de rotação e espelhamento só acontecem no treino. Os grupos de validação e teste ficam intactos para garantir uma avaliação justa.
- **Organização para o futuro:** Salvamos as configurações em um arquivo chamado `pipeline_info.json` e exportamos gráficos em PNG para usarmos nas próximas etapas do projeto.

6. Conclusão

A primeira parte do projeto foi concluída com sucesso. Conseguimos pegar um dataset real e robusto, padronizar o tamanho das imagens, equilibrar as cores e criar uma estrutura de dados sólida. Agora, o BloodMNIST está perfeitamente preparado para alimentar os modelos de inteligência artificial que vamos construir a seguir!